

Day 9: Benchmarking and profiling

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

- ▶ Days 3 to 8 gave many numbers: a latency, a model size, an accuracy, a frame rate. A number is useful only when a second person can measure it again and get the same value.
- ▶ One timed inference is not a measurement. The first inferences are slower than the later ones, the latency changes from run to run, and a mean hides the slow inferences. A fair benchmark has a warm-up, many repetitions, fixed inputs, and percentiles.
- ▶ A battery device has an energy budget. The energy of one inference and the duty cycle give the battery life. A hot processor reduces its clock frequency, so a short run gives a latency that a long run does not keep.
- ▶ In the lab you write the benchmark protocol first. Then you measure keyword spotting and image classification on the XIAOML Kit, and image classification and object detection on the Raspberry Pi 5. You write one report for the two boards.

After this day you can:

- ▶ Define latency percentiles, throughput, peak memory, and energy per inference.
- ▶ Design a fair benchmark with warm-up, repetitions, and fixed inputs.
- ▶ Measure one task on two boards and explain the differences.
- ▶ Explain thermal throttling and its effect on results.

Part 1 What to measure

Exercise: read one MLPerf Tiny result table and compare two devices

Part 2 Measurement method

Exercise: find three errors in a given benchmark script

Part 3 Power, energy, and temperature

Exercise: estimate the battery life of an always-on keyword spotting device

Which number is the latency?

MobileNetV2 on a Raspberry Pi 5	Mean	Median
float32, kernels with no optimization	2440 ms	2440 ms
float32, XNNPACK	11.24 ms	10.84 ms
int8, XNNPACK	3.91 ms	3.69 ms

- ▶ You saw this table on Day 7. Each number is correct: the same model, the same board, 50 runs for each row.
- ▶ “MobileNetV2 needs 4 ms on a Raspberry Pi 5” and “it needs 2.4 seconds” are the two ends of this table.
- ▶ The rows differ in the kernels and in the precision. The columns differ in the statistic.

Key point: A number is a result only together with its method: the device, the model file, the runtime, the input, the runs, and the statistic.

Source: *EdgeML with Raspberry Pi*, guide for ExecuTorch.

A benchmark has seven parts

Part	It fixes	Example
Task	What the system must do	Give the class of one image
Dataset	The inputs and the labels of the test	500 images of 10 classes
Model	The file that runs	MobileNetV2 in int8
Metrics	The numbers and their units	Median latency in ms, accuracy
Harness	The program that gives the inputs and reads the clock	A Python script on the board
System	The hardware and the software	Raspberry Pi 5, LiteRT, 4 threads
Run rules	How to run, and what you can change	20 runs to warm up, 200 timed runs

Key point: You can compare two results only when all seven parts are the same, or when exactly one part is different.

Three types of benchmark

System benchmark

Is the device
fast enough and
small enough?
latency, throughput,
memory, energy

Model benchmark

Is the model that you
deploy still good?
accuracy, mAP,
accuracy of each class

Data benchmark

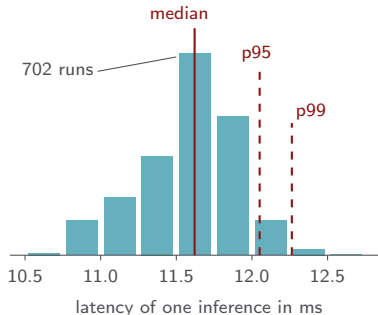
Do the test data
represent the product?
coverage, label quality,
separate test set

- ▶ Day 7 measured the system: one model in more than one runtime.
- ▶ Day 4 measured the model: the accuracy before and after the quantization.
- ▶ Day 8 measured the data: the effect of label errors on the mAP.

Key point: A product can pass one type and fail a different type. A report has the three types together.

Metric	Unit	Question
Latency	ms	How long does one inference take?
Throughput	inferences/s	How many inferences in one second?
Peak memory (RAM)	KB, MB	Does the model run together with the program?
Storage (flash)	KB, MB	Does the model file fit?
Energy per inference	μ J, mJ	How long does the battery supply the device?
Load time	ms, s	When is the first result ready after power-on?
Temperature	$^{\circ}$ C	Is the result the same after ten minutes?

- ▶ Each metric is a limit of the device from Day 1: time, memory, energy.
- ▶ The accuracy is not in this table. Each row needs it as a condition.

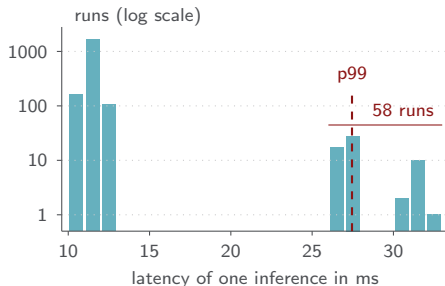


- ▶ 2000 inferences of MobileNetV2 with one thread and the same input.
- ▶ The fastest needs 10.62 ms, and the slowest needs 12.52 ms.
- ▶ **Median** (p50): one half of the inferences need this time or less. Here: 11.62 ms.
- ▶ **p95**: 95 percent need this time or less: 12.05 ms.
- ▶ **p99**: 99 percent: 12.26 ms.

Key point: One timed inference is one sample of this distribution. Report the median and one high percentile.

Source: An experiment of this course on an x86 processor.

A second program on the same core



ms	Alone	Shared
Mean	11.58	11.94
Median	11.62	11.46
p95	12.05	12.15
p99	12.26	27.45
Maximum	12.52	32.10

- ▶ The same 2000 inferences. A second program uses the core for 20 ms in each 400 ms, as a device that also reads a sensor.
- ▶ 58 inferences (2.9 percent) need more than 1.5 times the median. The mean changes by 3 percent. The p99 is 2.2 times as large.

Key point: The mean and the median hide the slow inferences. A high percentile shows them.

Source: An experiment of this course on an x86 processor.

Which statistic for which product

Statistic	It tells you	Use it for
Median	The typical inference	A comparison of two models or two runtimes
p95, p99	The time that 1 inference in 20, or 1 in 100, passes	A soft deadline: a voice command, a frame of a live video
Maximum	The slowest inference of this run, not of all runs	A first look at a hard deadline: a control loop
Mean	The total time divided by the number of inferences	The throughput and the energy of a long run

- ▶ MLPerf Inference reports the 90th percentile for queries of one sample, and the 99th percentile for queries of 8 samples.
- ▶ A percentile needs enough runs. The p99 of 100 runs is the second largest value: Part 2.

Key point: Write the statistic and the number of runs with each latency.

Source: The rules of MLPerf Inference (MLCommons).

Throughput is not always $1 / \text{latency}$

MobileNetV2 on 4 cores	Median latency	Throughput
One stream, 1 thread	11.49 ms	86.4 each second
One stream, 4 threads	3.26 ms	300.8 each second
Four streams, 1 thread each	12.43 ms	321.9 each second

- ▶ **Latency:** the time for one answer. **Throughput:** the answers in one second, for the complete device.
- ▶ One stream: the throughput is about $1 / \text{latency}$. Four threads make one inference 3.5 times as fast, not 4 times. Each row is a run of 10 seconds.
- ▶ Four streams at the same time give 7 percent more throughput. But each inference needs 3.8 times as long.
- ▶ One camera with a person in front of it needs a low latency. A recorder with four cameras needs a high throughput.

Key point: Decide which of the two numbers your product needs. Then set the threads and the streams for that number.

Source: An experiment of this course on an x86 processor.

MobileNetV2 in LiteRT	float32	int8
Model file (storage)	13.4 MB	3.9 MB
RAM that the load of the model adds	47.3 MB	24.6 MB
Peak RAM that the model adds	48.2 MB	25.0 MB

- ▶ The Python process with the runtime has 31 MB before the load.
- ▶ The runtime makes its own copy of the weights and adds the buffers of the activations. The RAM is 3.5 times the float32 file.
- ▶ Linux: read the peak of the process (VmHWM in /proc/self/status). A computer with 8 GB has room.
- ▶ Microcontroller: the flash holds the program and the model. The RAM holds the arena and the other data. No room: no run.

Key point: The file size does not tell you the RAM. Measure the peak RAM of the complete program on the device.

Source: An experiment of this course. 1 MB is 1024 KB.

Definition

$$E = P_{\text{active}} \times t_{\text{inference}}$$

$$1 \text{ mW} \times 1 \text{ ms} = 1 \mu\text{J}$$

A published result

Keyword spotting on a Cortex-M4 board:
43.19 ms and 1820.5 μJ for one inference.
Mean power: $1820.5/43.19 = 42.2 \text{ mW}$.

- ▶ The power (mW) says how fast the device uses its battery at this moment. The energy (μJ) says what one inference costs.
- ▶ A battery holds energy. The number of inferences of one battery is its energy divided by the energy per inference.
- ▶ A faster processor can use more power and less energy: it completes the work earlier. Speed and efficiency are two different results.
- ▶ A device that sleeps between two inferences also uses energy in the sleep: Part 3.

Source: MLPerf Tiny v1.2, board NUCLEO-L4R5ZI (MLCommons).
The latency and the power are calculations.

Model benchmark: test the file that you deploy

MobileNetV2 in LiteRT	Calibration	Accuracy
float32	none	77.4 percent
Weights in int8	none	77.6 percent
Full int8	100 images	75.2 percent
Full int8	100 images of noise	12.8 percent

- ▶ You know these rows from Day 7. The last two files have the same operators and the same precision. A speed test sees no difference.
- ▶ A latency with no accuracy rewards the wrong file: a model that gives a wrong answer wins each speed test.
- ▶ MLPerf gives each benchmark a quality target. One example: the accuracy must be 99 percent of the float32 accuracy.

Key point: The accuracy is a condition of each system result. Measure it with the file and the runtime of the device.

Source: An experiment of this course with 500 images.
Quality target: the rules of MLPerf Inference.

How many test samples?

Samples	Interval
100	± 8.2 points
200	± 5.8 points
500	± 3.7 points
1000	± 2.6 points
5000	± 1.2 points
50 000	± 0.4 points

$$\pm 1.96 \sqrt{\frac{p(1-p)}{N}}$$

- ▶ The 95 percent interval for an accuracy p of 77.4 percent and N test samples.
- ▶ 77.4 and 75.2 percent on 500 images: the difference of 2.2 points is smaller than the interval of one value.
- ▶ For ± 1 point you need 6720 samples.
- ▶ Two files on the same images: count the images for which the two answers differ. This comparison is more exact than two intervals.
- ▶ MLPerf Tiny tests the image classifier with 200 images. Its quality target (85 percent) is below the reference (86.5 percent).

Key point: A test set is a measuring instrument. Its size sets the smallest difference that you can see.

Source: A calculation with the normal approximation.
MLPerf Tiny: Banbury et al., 2021.

Data benchmark: the test data and the product

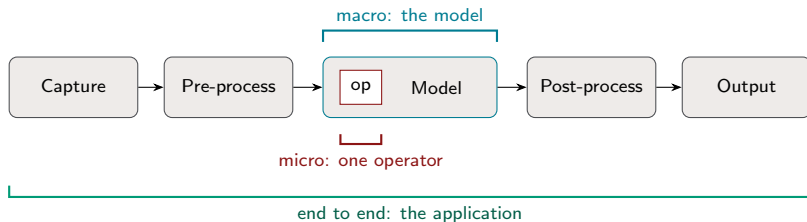
Question	Check	In this course
Coverage	Each condition of the product is in the test set: light, noise, speaker, sensor	Day 5: keywords in your own voice
Label quality	A second person checks a sample of the labels	Day 8: 10 percent of wrong classes cut mAP50 from 0.574 to 0.425
Separation	No test sample is in the training set	Day 8: mAP50 of 0.670 with training images, 0.546 with new images
Same source	The test data come from the sensor of the device	A photo on a screen is not the scene in front of the camera

- ▶ A study found label errors in the test sets of 10 common datasets: 3.3 percent or more on average, and 6 percent or more for ImageNet.

Key point: The best model and the fastest device cannot correct a test set that does not represent the product.

Source: Day 8: experiments of this course.
Label errors: Northcutt, Athalye, and Mueller, 2021.

What is in the timed window?



YOLO detector: time of one frame	Model	All steps	Part
x86 processor, 640 pixels (Day 8)	49.27 ms	52.75 ms	93 percent
Raspberry Pi Zero 2 W, 320 pixels	519.3 ms	1263.6 ms	41 percent

- The window of MLPerf Tiny is the model. The user of a product waits for all steps.

Key point: Name the window with each time: “model only” or “end to end”.

Source: x86: an experiment of this course.
Raspberry Pi Zero 2 W: *Edge AI Engineering*.

Five vendors, five claims

- ▶ A: “8 TOPS with int8”
- ▶ B: “15 TOPS with 4 bits”
- ▶ C: the latency of its own model
- ▶ D: a result of an old benchmark
- ▶ E: the peak throughput

You cannot compare two of them.

Suite	Devices	Main metrics
Training	Data centre	Time to a quality target
Inference, Datacenter	Servers	Throughput with a latency limit
Inference, Edge	Systems outside a data centre	Latency percentile, throughput
Tiny	Microcontrollers	Latency, accuracy, energy

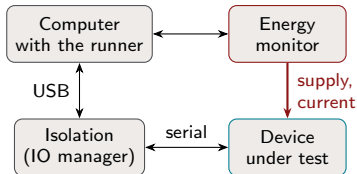
- ▶ A suite fixes the parts of the benchmark for all submitters: the task, the dataset, the model, the quality target, the harness, and the run rules.
- ▶ MLCommons, a non-profit group of companies and universities, makes the MLPerf suites. The submitters review the results of each other, and the code of each result is public.

Source: *Machine Learning Systems*, Volume I, chapter 12.

Task	Dataset and input	Model and file	Target
Keyword spotting	Speech Commands, 49×10	DS-CNN, 52.5 KB	90 percent
Visual wake words	Visual Wake Words, 96×96	MobileNetV1, 325 KB	80 percent
Image classification	CIFAR-10, 32×32	ResNet, 96 KB	85 percent
Anomaly detection	ToyADMOS, 5×128	Autoencoder, 270 KB	AUC 0.85

- ▶ The four tasks are typical work of a microcontroller. You trained the first two tasks on Day 5, with other models.
- ▶ Visual wake words: is a person in the image, yes or no?
- ▶ Anomaly detection: the sound of a machine. The model learns only the normal sound. Each model file is smaller than 350 KB.

Key point: A result is valid only when the model on the device reaches the quality target.



The setup for the energy. For the latency and the accuracy, the computer talks to the device directly.

- ▶ **Latency:** 5 runs. Each run has 10 seconds or more and 10 inferences or more. The result is the median of the inferences each second.
 - ▶ **Accuracy:** one inference for each test input, on the device.
 - ▶ **Energy:** the same 5 runs, with the energy monitor. The result is the median in μJ for each inference.
 - ▶ **Window:** the model only. The feature extraction is not in the time.
- ▶ **Closed division:** the reference model and dataset. Post-training quantization is permitted, new training is not.
 - ▶ **Open division:** a different model or training is permitted. The submitter documents each change.

Source: Banbury et al., "MLPerf Tiny Benchmark", 2021.

NUCLEO-L4R5ZI: Arm Cortex-M4 at 120 MHz, 1.8 V, inference software X-CUBE-AI v9.0, closed division.

Benchmark	Inferences/s	Published		Calculated	
		Accuracy	Energy	Latency	Power
Visual wake words	11.245	85.2	3894.6 μ J	88.9 ms	43.8 mW
Image classification	6.056	87.0	6919.4 μ J	165.1 ms	41.9 mW
Keyword spotting	23.155	90.2	1820.5 μ J	43.2 ms	42.2 mW
Anomaly detection	190.086	0.86	203.4 μ J	5.3 ms	38.7 mW

- ▶ Latency = 1000 / rate. Mean power = energy $\times$ rate.
- ▶ Each accuracy is above its quality target: 80, 85, 90 percent, and 0.85.
- ▶ The power is almost the same for the four models. The processor is active all the time, so the energy follows the latency.
- ▶ Read the system description first: the processor, the clock, the voltage, and the software are a part of the result.

Source: MLPerf Tiny v1.2, closed division (MLCommons):
github.com/mlcommons/tiny_results_v1.2

Keyword spotting, closed division	Latency	Energy	Accuracy
Cortex-M4 at 120 MHz, 1.8 V	43.2 ms	1820.5 μ J	90.2
Accelerator at 30.7 MHz, 0.9 V	4.4 ms	31.5 μ J	91.1
Accelerator at 98.7 MHz, 1.1 V	1.5 ms	43.8 μ J	91.1

- ▶ The accelerator (NDP9120) has hardware for the layers of a neural network. At 0.9 V it is 10 times as fast as the microcontroller (NUCLEO-L4R5ZI) and uses 58 times less energy.
- ▶ The same chip with a higher voltage and a faster clock: 2.9 times as fast, with 39 percent more energy for each inference.
- ▶ The two accuracy values differ: each submitter quantizes the reference model with its own tools.

Key point: The voltage and the clock are settings of the benchmark. One chip gives more than one result.

Source: MLPerf Tiny v1.2, closed division (MLCommons).
The latency is a calculation.

Scenario	How the queries arrive	Result
Single stream	The next query starts when the last query is complete. One sample in each query.	The 90th percentile of the latency
Multistream	The same, with 8 samples in each query: 8 cameras.	The 99th percentile of the latency
Offline	All samples are there at the start.	The throughput in samples each second

- ▶ A run has 600 seconds or more. A long run includes the effect of the temperature (Part 3).
- ▶ Two of its tasks: image classification with ResNet-50 on ImageNet, and object detection with a YOLO model on 1525 images of COCO.
- ▶ Quality target of the classifier: 99 percent of the float32 accuracy of 76.46 percent.
- ▶ The Datacenter category has the scenario “Server”: the queries arrive at random times.

Source: The rules of MLPerf Inference (MLCommons), 2026-10-02.
The tasks change with each round.

What a standard benchmark does not tell you

The suite uses	Your product has
Its reference model	Your model, with your classes
Its dataset	Your users, your sensor, your room
The time of the model only	Capture, features, and post-processing
Inferences with no pause	A duty cycle with sleep
The best software of the submitter	Your runtime and your settings
A laboratory	A closed box, in the sun

- ▶ Use a published result to select a chip or a runtime.
- ▶ Use your own benchmark for the decisions of your product. It has the same seven parts.
- ▶ Part A of the lab: you write these parts as a protocol before the first measurement.

Key point: A benchmark is a model of the use, not the use. Say what your benchmark does not include.

Exercise: read an MLPerf Tiny result table

Two boards of one submitter in MLPerf Tiny v1.2, closed division, with the same software. A: Cortex-M33 at 160 MHz. B: Cortex-M7 at 280 MHz.

Board	Benchmark	Inferences/s	Accuracy	Energy
A	Visual wake words	16.870	85.2	1354.7 μ J
A	Image classification	9.003	87.0	2579.0 μ J
B	Visual wake words	42.009	85.2	2965.5 μ J
B	Image classification	24.483	87.0	5090.3 μ J

1. Calculate the latency of the visual wake words model on each board.
2. Which board is faster, and by which factor? Which board uses less energy for one inference, and by which factor?
3. Calculate the mean power of each board during this benchmark.
4. A door camera with a battery looks for a person 5 times each second. Which board do you select? Which number is missing?
5. Why is the accuracy the same on the two boards?

Answer: read an MLPerf Tiny result table

Visual wake words	Latency	Energy	Mean power
A: Cortex-M33, 160 MHz	59.3 ms	1354.7 μ J	22.9 mW
B: Cortex-M7, 280 MHz	23.8 ms	2965.5 μ J	124.6 mW

1. $1000/16.870 = 59.3$ ms and $1000/42.009 = 23.8$ ms.
2. B is 2.5 times as fast. A uses 2.2 times less energy for one inference. The image classifier gives almost the same factors: 2.7 and 2.0.
3. Energy $\times$ rate: 22.9 mW for A and 124.6 mW for B. B uses 5.5 times the power for 2.5 times the speed.
4. Board A. The limit is 200 ms, and the two boards are fast enough. A uses less than one half of the energy for each inference. The table does not give the power in the sleep between two inferences: Part 3.
5. The two boards run the same int8 model with the same software. In the closed division, the model is the reference model.

Part 1 What to measure

Exercise: read one MLPerf Tiny result table and compare two devices

Part 2 Measurement method

Exercise: find three errors in a given benchmark script

Part 3 Power, energy, and temperature

Exercise: estimate the battery life of an always-on keyword spotting device

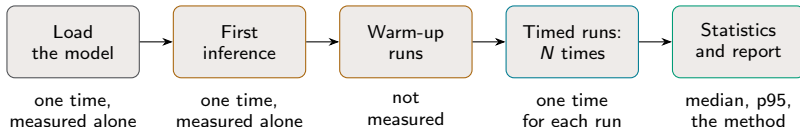
Two students, one model, two numbers

	Method	Result
Student A	Start the program. Run the model 5 times. Report the mean.	13.35 ms
Student B	Start the program. Run the model 20 times with no measurement. Then report the median of the next 10 runs.	10.95 ms

- ▶ The same model file, the same computer, the same process, the same input.
- ▶ The result of student A is 22 percent higher.
- ▶ On Day 7, two runtimes differed by less than 5 percent for this model. The method changes more.
- ▶ The two numbers are correct. They answer two different questions.

Key point: The method can change a result more than the optimization that you want to measure. So the method is a part of the result.

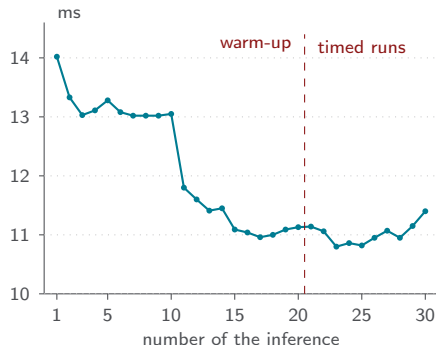
Source: An experiment of this course on an x86 processor.



- ▶ One fixed input, ready before the clock starts.
- ▶ The clock is around one inference. Nothing else is in the window: no print, no file, no copy that the product does not need.
- ▶ Keep each time. Calculate the statistics after the loop.
- ▶ Read the temperature between the runs, not in a run.
- ▶ This is the loop of the sketch and of the script of the lab.

Key point: Each number of a report needs four facts: the warm-up runs, the timed runs, the input, and the statistic.

Warm-up: the first inferences are slower



- ▶ The first 30 inferences of a new process.
- ▶ Inferences 1 to 10 need about 13 ms. From inference 15, the time is about 11 ms.
- ▶ A processor can increase its clock when the load starts.
- ▶ The caches and the memory pages fill at the first runs.
- ▶ A runtime prepares some buffers at the first inference.

Key point: Plot the first times. The warm-up ends when the times are stable. Here: 20 runs.

Source: An experiment of this course on an x86 processor.

YOLO on a Raspberry Pi Zero 2 W, the time of each step:

	Pre-process	Inference	Post-process	Sum
First run (cold)	289.7 ms	2545.1 ms	1540.9 ms	4375.7 ms
Second run (warm)	237.6 ms	519.3 ms	506.7 ms	1263.6 ms

- ▶ The same command two times, with no change of the model.
- ▶ The first inference needs 4.9 times as long. The complete first run needs 3.5 times as long.
- ▶ The first run loads the code of the libraries and reads memory pages from the microSD card. The second run finds them in the RAM.
- ▶ A board with little RAM and slow storage has the largest effect.

Key point: Do not mix the first run with the other runs. Measure it alone, or do not report it.

Source: *Edge AI Engineering*, chapter on YOLO.

MobileNetV2 in LiteRT, 10 new processes	Median
Load the model and make the interpreter	96.9 ms
First inference	11.87 ms
Inferences 21 to 30	11.08 ms

- ▶ A camera that runs all the time: the steady time of 11.08 ms is the latency of the product.
- ▶ A sensor that wakes, makes one inference, and sleeps again: each inference is a first inference. The product pays the load also: $96.9 + 11.87 = 108.8$ ms.
- ▶ The second product is about 10 times slower than the benchmark of the first product says.

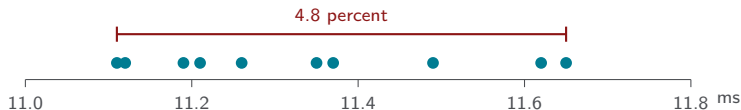
Key point: Report three times: the load, the first inference, and the steady latency. Your product decides which one counts.

Source: An experiment of this course on an x86 processor.

Source	Effect	What you do
Other programs on the device	Some inferences wait for a core	Close them. Report a high percentile.
The clock of the processor changes	Slow first runs, slow runs when the chip is hot	Warm up. Record the temperature and the clock.
The input	A detector with many objects needs more post-processing	Use fixed inputs
Interrupts on a microcontroller	Wi-Fi and timers stop the inference for a short time	Stop the radio, or report it as active
The operating system moves a thread	The caches are cold after the move	More runs. On Linux: fix the cores.

Key point: You cannot remove the variance. You can control it, measure it, and report it.

Repeat the benchmark: ten runs, ten medians



- ▶ Ten times the same benchmark: a new process, 20 warm-up runs, 200 timed runs, the median.
- ▶ The lowest median is 11.11 ms, and the highest is 11.65 ms.
- ▶ A second model that is 3 percent faster in one run of the benchmark is not faster. The difference is inside the noise.
- ▶ MLPerf Tiny repeats each test five times and reports the median of the five results.

Key point: Run the complete benchmark more than one time. The difference between two runs is the smallest gain that you can report.

Source: An experiment of this course on an x86 processor.
MLPerf Tiny: Banbury et al., 2021.

How many timed runs?

Runs	Lowest median	Highest median
5	10.88 ms	12.37 ms
20	11.06 ms	12.08 ms
50	11.21 ms	11.93 ms
200	11.34 ms	11.79 ms

- ▶ 2000 latencies of one process, cut into groups of 5, 20, 50, or 200 runs.
- ▶ Each group gives one median. The table shows the two extreme groups.
- ▶ With 5 runs, the median can be 10.88 ms or 12.37 ms: a range of 13 percent. With 200 runs the range is 4 percent.
- ▶ More runs give a more stable median. The gain becomes small: 10 times the runs do not give 10 times the stability.
- ▶ On a slow device, set a time limit also: 200 runs, or 30 seconds.

Key point: Select the number of runs for the difference that you want to see. Write the number in the report.

Source: An experiment of this course on an x86 processor.

Do not report the best run

2000 inferences, a second program on the core	Latency
Minimum (“the best run”)	10.61 ms
Median	11.46 ms
Mean	11.94 ms
99th percentile	27.45 ms

- ▶ The minimum is the time with no disturbance. A product has disturbances. The best of three runs is not a benchmark result.
- ▶ The mean of few runs is not stable: for groups of 5 runs of this data, the mean is between 10.85 ms and 16.11 ms.

Key point: Report the median for the typical case and a high percentile for the slow case. Do not select the run that you like.

Source: An experiment of this course on an x86 processor.

A percentile needs many runs

Runs	Lowest p99	Highest p99
20	11.44 ms	32.10 ms
50	26.50 ms	32.10 ms
200	27.03 ms	31.27 ms

- ▶ The same data as before: 3 percent of the inferences are slow.
- ▶ A group of 20 runs can have no slow inference. Its p99 is then 11.44 ms.
- ▶ The rule of the rank: the p99 of N runs is the value with the rank $0.99 \times N$, rounded up. For 20 runs, this is the maximum.
- ▶ To see an event that occurs one time in 100 runs, you need many times 100 runs.
- ▶ MLPerf Inference uses 24 576 queries for a 90th percentile, and 270 336 queries for a 99th percentile.

Key point: With 20 runs, report the median and the maximum. Do not call the maximum a p99.

Source: An experiment of this course on an x86 processor.
Query numbers: the rules of MLPerf Inference.

Clock	Use it?	Reason
<code>time.perf_counter_ns()</code>	Yes, in Python	It does not go back, and it has a small step
<code>time.time()</code>	No	It is the clock of the day. The system can set it during your run.
<code>micros()</code>	Yes, in a sketch	A step of microseconds
<code>millis()</code>	No	A step of 1 ms: an inference of 3.4 ms shows 3 or 4

- ▶ A clock read has a cost. On the work computer, one read of `perf_counter_ns()` needs 146 ns.
- ▶ The window must be much longer than the step and the cost of the clock. For a window of some microseconds, time 1000 runs with one pair of clock reads.
- ▶ On a microcontroller you can also set a pin before and after the inference, and measure the pulse with an oscilloscope.

Source: The cost of a clock read: an experiment of this course.

```
t0 = time.perf_counter_ns()
frame = camera.capture_array()           # capture
x = preprocess(frame)                   # resize, normalize
t1 = time.perf_counter_ns()
interpreter.set_tensor(input_index, x)
interpreter.invoke()                     # the model
y = interpreter.get_tensor(output_index)
t2 = time.perf_counter_ns()
boxes = postprocess(y)                  # decode, suppress
t3 = time.perf_counter_ns()
model_ms.append((t2 - t1) / 1e6)
pipeline_ms.append((t3 - t0) / 1e6)
```

- ▶ Two windows in one loop. `model_ms` compares runtimes, precisions, and thread counts. `pipeline_ms` is the time that the user waits.
- ▶ The frame rate of a product is 1000 divided by the pipeline time, not by the model time.
- ▶ MLPerf Tiny reports the model only. The lab of Day 8 reported the pipeline. The lab of today reports the model.

Key point: Give each time the name of its window. A comparison of a model time with a pipeline time is not valid.

Median of the window	<code>invoke()</code>	With the copies	Difference
MobileNetV2, float32	11.281 ms	11.384 ms	0.9 percent
Keyword model, int8	0.2468 ms	0.2489 ms	0.8 percent

- ▶ The two copies: `set_tensor()` writes the input into the model, and `get_tensor()` reads the output.
- ▶ Here the copies cost less than 1 percent: 103 microseconds for an image of 224 pixels, and 2 microseconds for the small input of the keyword model.
- ▶ A product needs the copies. So the window of the lab script has them.
- ▶ On a different runtime, the cost of the copies can be larger. Measure it one time, and write which window you use.

Key point: “Model only” has more than one meaning. Write the first call and the last call of your window in the protocol.

Source: An experiment of this course on an x86 processor, one thread.

A faster model in a slow pipeline



- ▶ The warm run of the last table: the model has 519.3 ms of 1263.6 ms. This is 41 percent.
- ▶ A model that is 2 times as fast makes the pipeline 1.26 times as fast. A model with no time at all gives 1.70 times: this is the limit.
- ▶ The law of Amdahl (Day 7): with the part f of the time in the model, and a model that is s times as fast, the gain is $1/((1 - f) + f/s)$.

Key point: Measure the pipeline first. Then optimize its largest step. This step is not always the model.

Source: Times: *Edge AI Engineering*, chapter on YOLO.
The two faster cases are calculations.

Model type	Does the time depend on the input?	Input for a benchmark
Image classifier, keyword model	No. Each input needs the same operations.	One fixed input is sufficient
Detector	Yes, in the post-processing: more candidates need more time	A fixed set of images of the product
Language model	Yes: the time grows with the number of tokens	Fixed prompts and a fixed output length

- ▶ Day 8: the suppression of 47 candidates needed 1.4 ms, and the suppression of 727 candidates needed 43 ms. Same model, same image, a different score threshold.
- ▶ Use the same inputs, in the same order, for each system that you compare.
- ▶ Random numbers are a valid input for a classifier. Fix the seed.

Key point: An input that changes from run to run adds variance that is not a property of the system.

Source: Day 8: an experiment of this course.

The same loop on a microcontroller

```
for (int i = 0; i < kTimedRuns; i++) {  
    bench_input(bench, input);           // fill the input: not in the window  
    const unsigned long start = micros();  
    interpreter.Invoke();                 // the window: the model only  
    times_us[i] = micros() - start;  
}  
sortTimes(times_us, kTimedRuns);         // statistics after the loop  
Serial.printf("median %lu us\n", percentileUs(times_us, kTimedRuns, 50));
```

- ▶ No `Serial.print` in the window. On a serial line with 115 200 baud, a line of 40 characters needs 3.5 ms.
- ▶ Fill the input before each inference. The interpreter can use the memory of the input tensor again for a later tensor.
- ▶ The result also depends on the memory of the arena (internal RAM or PSRAM), on the clock of the processor, and on the radio. Write the three settings in the report.

Key point: The rules are the same on each device: warm up, repeat, keep each time, and print after the loop.

Source: The sketch `kit_bench` of the lab.

Item	XIAOML Kit	Raspberry Pi 5
Board and processor	XIAO ESP32S3, 240 MHz	Raspberry Pi 5, 4 cores at 2.4 GHz
Software	esp32 core and library, with versions	Operating system, Python, runtime, with versions
Model file	Name, bytes, precision	Name, bytes, precision
Settings	PSRAM, arena memory, clock	Threads, delegate, clock governor
Power and cooling	USB or battery	Power supply, active cooler
State during the run	Radio on or off, chip temperature	Other programs, temperature, throttle state

- ▶ Day 7: the same model needed 2440 ms or 10.84 ms on the same board. The difference was one line of this table: the kernels.
- ▶ The script of the lab prints most of these items. Copy them into the report.

Key point: A second person must be able to build your system again from this table.

Error	Result
No warm-up	The cold start is in the mean
Few runs, and the mean	One slow run moves the result
The best run	A number that the product does not reach
The load or a print in the window	You measure the storage or the terminal
A new input in each run	More variance, and no comparison with a second system
A different window for two systems	A model time against a pipeline time
No thread setting	The runtime selects a number that you do not know
A latency with no accuracy	The fast model is possibly wrong
A short run on a hot or cold chip	The result changes after some minutes (Part 3)

Key point: Each error gives a number that looks exact. Only the written method shows the error.

The protocol: write it before you measure

1. The question: which decision does the number support?
 2. The seven parts: task, dataset, model, metrics, harness, system, run rules.
 3. The window: the first call and the last call.
 4. The warm-up runs and the timed runs, with a time limit.
 5. The statistics: the median, one high percentile, and the first inference.
 6. The number of repetitions of the complete benchmark.
 7. Your prediction for each result, with a reason.
- ▶ A protocol that you write after the measurement follows the result that you like.
 - ▶ A change during the work is permitted. Write the change and its reason.
 - ▶ Part A of the lab: 20 minutes for this protocol, with one table for the two boards.

Key point: A result is useful when a second person can repeat it. The protocol is the instruction for that person.

Exercise: find three errors in a benchmark script

```
import time
import numpy as np
from ai_edge_litert.interpreter import Interpreter

best = 1e9
for i in range(5):
    start = time.time()
    interpreter = Interpreter(model_path="mnv2.tflite")
    interpreter.allocate_tensors()
    image = np.random.rand(1, 3, 224, 224).astype("float32")
    interpreter.set_tensor(interpreter.get_input_details()[0]["index"], image)
    interpreter.invoke()
    print("run", i, "is complete")
    best = min(best, time.time() - start)
print("Latency of MobileNetV2: %.1f ms" % (1000 * best))
```

The script prints one number. Find three errors or more in the method. For each error, say how it changes the number. Then write the list of facts that the last line must print.

Answer: find three errors in a benchmark script

```
interpreter = Interpreter(model_path="mnv2.tflite", num_threads=4)      # 6
interpreter.allocate_tensors()                                          # 1
index = interpreter.get_input_details()[0] ["index"]
rng = np.random.default_rng(0)
image = rng.random((1, 3, 224, 224), dtype=np.float32)               # 3
for _ in range(20):                                                  # 2
    interpreter.set_tensor(index, image)
    interpreter.invoke()
times = []
for _ in range(200):                                                  # 2
    start = time.perf_counter_ns()                                     # 5
    interpreter.set_tensor(index, image)
    interpreter.invoke()
    times.append((time.perf_counter_ns() - start) / 1e6)              # 4
```

(1) The load is in the window: each run is a cold start. (2) No warm-up, and only 5 runs. (3) A new random input in each run, made in the window. (4) A print in the window, and the minimum as the result. (5) The clock of the day. (6) No thread setting. The last line must give: the median and a high percentile, the 20 warm-up runs, the 200 timed runs, the input, the window, the threads, and the device.

Part 1 What to measure

Exercise: read one MLPerf Tiny result table and compare two devices

Part 2 Measurement method

Exercise: find three errors in a given benchmark script

Part 3 Power, energy, and temperature

Exercise: estimate the battery life of an always-on keyword spotting device

How long does the battery supply the device?

Keyword model on a Cortex-M33 board	Published
Latency of one inference	29.1 ms
Energy of one inference	656.2 μ J

- ▶ A product makes one inference in each second. Its battery has 1000 mAh at 3.7 V.
- ▶ The inferences alone use 0.66 mW. The battery has energy for 235 days of inferences.
- ▶ No product gets this time. The device waits 971 ms in each second, and the wait also needs power.
- ▶ A board that waits with 157 mW empties the same battery in less than one day.

Key point: The latency says how long the processor works. The battery life depends on all 1000 ms of each second.

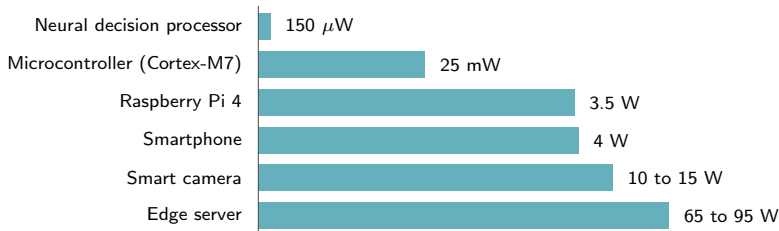
Source: MLPerf Tiny v1.2, board NUCLEO-U575ZI-Q (MLCommons).
The latency and the battery values are calculations.

Quantity	Formula and unit	It tells you
Power	$P = V \times I$, in W	How fast the device uses energy at this moment
Energy	$E = P \times t$, in J	What one task costs: one inference, one day
Capacity	mAh	The charge of a battery. With its voltage: the energy.

- ▶ 1 J = 1 W for 1 s. 1 mW for 1 ms is 1 μ J. 1 Wh = 3600 J.
- ▶ A battery with 1000 mAh at 3.7 V has $1 \text{ Ah} \times 3.7 \text{ V} = 3.7 \text{ Wh}$. This is 13 320 J.
- ▶ Battery life in hours = energy of the battery in Wh, divided by the mean power in W.
- ▶ 3.7 Wh and a mean power of 0.217 W give 17 hours.

Key point: A data sheet gives a current in mA. Multiply it by the supply voltage to get the power.

The power range of edge devices



- ▶ The bars have a logarithmic scale. From the first row to the last row, the power grows by a factor of about 500 000.
- ▶ The boards of this course: the ESP32-S3 chip uses about 0.2 W when it calculates, and the Raspberry Pi 5 uses 3 W or more.
- ▶ One measuring instrument cannot cover this range. A device for microamperes and a device for amperes are two different tools.

Key point: Before you measure, estimate the range: microwatts, milliwatts, or watts. Then select the instrument.

Source: *Machine Learning Systems*, Volume I, chapter 12.

For a range, the bar shows the centre.

Where the power of a chip goes

Two parts

$$P = \alpha C V^2 f + I_{\text{leak}} V$$

Switching: it grows with the clock f and with the square of the voltage V .

Leakage: it is there with no work, and it grows with the temperature.

Accelerator	0.9 V	1.1 V
Clock	30.7 MHz	98.7 MHz
Latency	4.4 ms	1.5 ms
Mean power	7.2 mW	29.6 mW
Energy	31.5 μ J	43.8 μ J

- ▶ The same chip of Part 1. With the higher voltage and the faster clock, it is 2.9 times as fast and uses 4.1 times the power.
- ▶ The energy of one inference is 39 percent higher. The higher voltage has a cost for each operation.
- ▶ A faster clock needs a higher voltage. So the power grows faster than the speed.

Key point: Speed costs power. A chip that runs slower at a lower voltage uses less energy for the same work.

Source: MLPerf Tiny v1.2, keyword model (MLCommons).
The latency and the power are calculations.

State of the chip, at 3.3 V	Current	Power
Wi-Fi transmits (peak)	340 mA	1122 mW
240 MHz, one core calculates, the other core waits	65.9 mA	217 mW
160 MHz, one core calculates	54.6 mA	180 mW
80 MHz, one core calculates	42.6 mA	141 mW
240 MHz, the two cores wait	47.6 mA	157 mW
Light sleep	0.24 mA	0.79 mW
Deep sleep	0.008 mA	0.03 mW

- ▶ A chip that waits at 240 MHz uses 72 percent of the active power. To wait is not to sleep. The light sleep uses 275 times less power.
- ▶ The radio is off in the rows 2 to 7. The values are typical values with the clocks of the peripherals on.

Key point: Three numbers decide most battery estimates: the active power, the sleep power, and the time in each state.

Source: Data sheet of the ESP32-S3, version 2.2 (Espressif Systems).
The power is a calculation.

A slower clock does not save energy

Clock	Current	Time	Current $\times$ time	Factor
240 MHz	65.9 mA	$1.0 \times t$	$65.9 \times t$	1.00
160 MHz	54.6 mA	$1.5 \times t$	$81.9 \times t$	1.24
80 MHz	42.6 mA	$3.0 \times t$	$127.8 \times t$	1.94

- ▶ The table has one assumption: the time of an inference is proportional to $1 / \text{clock}$. You measure this in the lab.
- ▶ The voltage is the same in each row, so the energy follows the current multiplied by the time.
- ▶ At 80 MHz the current is 35 percent lower. But the inference needs 3 times as long, so its energy is almost 2 times as large.
- ▶ A large part of the current does not depend on the clock. The better plan: calculate at the full clock, then sleep.

Key point: Low power is not low energy. Compare the energy of the complete task.

Source: Calculations with the data sheet of the ESP32-S3.

The boundary: chip, board, product

Product: battery, case

Board: regulator, light,
camera, card

Chip: processor,
memory, radio

	Chip	XIAO board	Sense board
Light sleep	0.79 mW	7.6 mW	19.0 mW
Deep sleep	0.03 mW	0.05 mW	11.4 mW

Chip: data sheet, 3.3 V. Boards: values of the vendor at 3.8 V.

- ▶ The board of the kit, with its camera and its card slot, uses 24 times the light sleep power of the chip. In deep sleep the factor is more than 400.
- ▶ A battery estimate with the chip value is too good by this factor.
- ▶ MLPerf Tiny measures the power of the core supply only. A USB power meter measures the complete board.

Key point: Say where your power value comes from: the chip, the board, or the product. Compare only values with the same boundary.

Source: Data sheet of the ESP32-S3.
Page of the XIAO ESP32S3 boards (Seeed Studio).

Four ways to get a power value

Method	It gives	Limit
Data sheet value	A typical current for each state of the chip	The chip only, and not your program
USB power meter between the supply and the board	Voltage, current, and power of the complete board	Slow, and not exact enough for a sleep current
Energy monitor, or a small resistor in the supply line with an oscilloscope	The current many times in each second: one inference is visible	More equipment and more work
Sensors of the board (Raspberry Pi 5)	The voltage and the current of each supply rail	An estimate. Compare it with a meter one time.

- ▶ MLPerf Tiny uses an energy monitor that supplies the device and measures the energy of the timed window.
- ▶ The lab uses a USB power meter if the classroom has it. If not: the data sheet for the kit, and the sensors for the Raspberry Pi.

Key point: Each method is valid if the report names it. A value with no method is not valid.

From the power and the latency

$$E = P_{\text{active}} \times t$$

Read the power during a long run.
Multiply it by the median latency.

Example: 217 mW and 50 ms give
10.85 mJ.

From the energy of many inferences

$$E = E_{\text{window}} / N$$

Measure the energy of a window with
 N inferences, and divide.

This is the method of MLPerf Tiny:
10 seconds or more for each window.

- ▶ The first method needs only a slow meter. The second method needs a meter that adds the energy.
- ▶ Decide what you do with the idle power. “Total”: the complete power during the inference. “Added”: the power above the idle state. Write which one you report.
- ▶ Use the same window as for the latency: the model only, or the pipeline.
- ▶ Warm up first. The power changes with the temperature.

Source: MLPerf Tiny: Banbury et al., 2021.

A faster board is not always more efficient

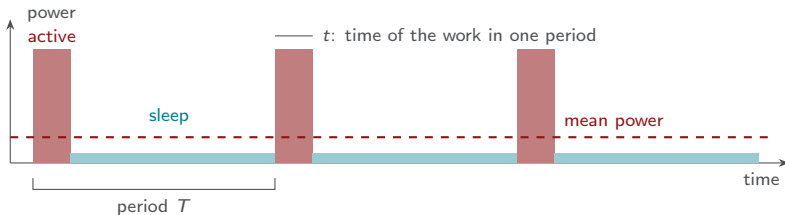
Power of the Raspberry Pi 5	Value	Divided by 217 mW
Published example with no load	3.2 W	15
Moderate load, lower limit	5 W	23
Moderate load, upper limit	7 W	32

- ▶ The same model runs on the two boards. The Raspberry Pi is much faster, and it uses much more power.
- ▶ With 5 W against 217 mW, the Raspberry Pi uses less energy for one inference only if it is more than 23 times as fast.
- ▶ For a small model, the fixed power of the large board is the problem: it pays 3 W also when it does nothing.
- ▶ In the lab you measure the speed factor for four models. Then you know the answer for each of them.

Key point: Energy per inference = power $\times$ latency. A board wins only if its speed gain is larger than its power factor.

Source: Raspberry Pi 5: *Edge AI Engineering*, chapter “Setup”.
ESP32-S3: its data sheet.

The duty cycle



$$\alpha = \frac{t}{T} \quad P_{\text{mean}} = \alpha P_{\text{active}} + (1 - \alpha) P_{\text{rest}}$$

- ▶ α is the duty cycle: the part of the time in which the device works.
- ▶ P_{rest} is the power of the state between two tasks: the idle state, the light sleep, or the deep sleep.
- ▶ A camera with a frame rate has α near 1. A sensor that reports one value in each minute has α near 0.

Key point: The mean power, not the peak power, empties the battery.

From the mean power to the battery life

One inference of 50 ms in each second: a duty cycle of 5 percent. Active power 217 mW. Battery 3.7 Wh.

State between two inferences	Rest power	Mean power	Battery life
The chip waits (idle)	157 mW	160.0 mW	23 hours
Light sleep, the chip only	0.79 mW	11.6 mW	13.3 days
Light sleep, the Sense board	19.0 mW	28.9 mW	5.3 days

- ▶ Mean power: $0.05 \times 217 + 0.95 \times 0.79 = 11.6$ mW. Battery life: $3700/11.6 = 319$ hours.
- ▶ The same model, the same latency, the same battery: from 23 hours to 13 days.
- ▶ The third row uses the board value of the vendor. It is the row that a real product gets.

Key point: Decide the rest state first. An optimization of the model changes only the first term of the formula.

The rest state decides the battery life

Duty cycle	Mean power	Battery life	Part of the sleep in the energy
100 percent	217 mW	17 hours	0 percent
10 percent	22.4 mW	6.9 days	3 percent
1 percent	2.95 mW	52 days	26 percent
0.1 percent	1.01 mW	153 days	78 percent

- ▶ Active power 217 mW, light sleep of the chip 0.79 mW, battery 3.7 Wh.
- ▶ From 100 to 10 percent, the battery life grows by a factor of almost 10. From 1 to 0.1 percent, it grows only by a factor of 3.
- ▶ At a low duty cycle, the sleep power is the limit. A faster model then gives almost no gain.
- ▶ With the light sleep of the Sense board (19 mW) and a duty cycle of 1 percent, the sleep has 90 percent of the energy.

Key point: Find the larger term first: αP_{active} or $(1 - \alpha) P_{\text{rest}}$. Optimize that term.

Source: Calculations with the data sheet of the ESP32-S3 and the value of the vendor for the board.

The floor: energy times rate

Keyword model	Energy	One inference each second
Cortex-M4 at 120 MHz	1820.5 μJ	1.82 mW
Cortex-M7 at 280 MHz	1375.6 μJ	1.38 mW
Cortex-M33 at 160 MHz	656.2 μJ	0.66 mW
Neural accelerator at 0.9 V	31.5 μJ	0.03 mW

- ▶ Energy per inference $\times$ inferences in each second = the mean power of the inferences. 1 μJ in each second is 1 μW .
- ▶ This is the lowest mean power that the device can have at this rate. The rest power comes on top.
- ▶ An always-on keyword detector makes about 4 inferences in each second. It also needs the microphone and the features all the time.
- ▶ So a product uses two stages (Day 5): a very small detector that runs all the time, and a larger model that starts on demand.

Source: MLPerf Tiny v1.2, closed division (MLCommons).
The power is a calculation.

The radio can cost more than the model

ESP32-S3 at 3.3 V	Current	Power
One core calculates, radio off	65.9 mA	217 mW
Wi-Fi receives	88 mA	290 mW
Wi-Fi transmits (peak)	340 mA	1122 mW

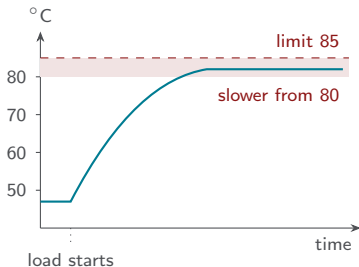
- ▶ The peak of a transmission is 5.2 times the power of the processor at work.
- ▶ A device that sends each sensor value to a server keeps its radio active. A device that decides locally sends only the events.
- ▶ This is a reason for inference on the device (Day 1), and the topic of Day 12: local decisions and MQTT.
- ▶ A benchmark of the model only does not show this cost. Add the radio to the energy budget of the product.

Key point: The energy budget of a product has three parts: the sensor, the model, and the communication.

Source: Data sheet of the ESP32-S3, version 2.2.

The power is a calculation.

Heat: why a chip becomes slower



- ▶ All the electric power of a chip becomes heat. The temperature grows until the heat that leaves is equal to the power.
- ▶ Near its limit, the board decreases the clock of the processor: **thermal throttling**. The latency grows, with no change of the program.

- ▶ Raspberry Pi: between 80 and 85 °C, the clock of the cores goes down in steps. The limit is 85 °C.
- ▶ The cooler of the Raspberry Pi 5 starts at 50 °C and has its full speed at 75 °C.

Key point: A hot chip is a slower chip. The temperature is a part of the system description.

Source: The limits: the documentation of the Raspberry Pi.

Read the thermal state of the Raspberry Pi

```
vcgencmd measure_temp           # temp=47.2'C  
cat /sys/class/thermal/thermal_zone0/temp # 47200: in 0.001 degrees  
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_cur_freq # clock in kHz  
vcgencmd get_throttled          # throttled=0x0
```

Value	Now	Value	Since the start
0x1	The supply voltage is too low	0x10000	The voltage was too low
0x2	The clock of the cores is limited	0x20000	The clock was limited
0x4	The board throttles	0x40000	The board throttled
0x8	The soft temperature limit is active	0x80000	The soft limit was active

- ▶ 0x0: no event since the start of the board. 0x50000: the voltage was too low, and the board throttled, in the past.
- ▶ A weak power supply also limits the clock. Use the 27 W supply.

Key point: Read these values before and after each benchmark. The script of the lab stores them with each result.

Source: The documentation of the Raspberry Pi.
The example temperature: *Edge AI Engineering*.

The method

`vcgencmd pmic_read_adc` prints the voltage and the current of each supply rail.

Sum of all rails: $\sum V_i \times I_i$

Estimate for the board:

$1.1451 \times \text{sum} + 0.5879 \text{ W}$

State	°C	W
No load	40 to 50	3.0 to 3.5
Moderate load	55 to 70	5 to 7
Heavy load	70 to 80	7 to 10

- ▶ The published example with no load: a sum of 2.312 W, an estimate of 3.235 W, and 47 °C.
- ▶ The sum is the power of the main chips. The correction adds the rest of the board. It is an estimate of the author of the book.
- ▶ USB devices and the loss of the power supply are not in the value.
- ▶ The lab script reads this estimate between the timed runs.

Key point: An estimate with a known method is useful. Compare it one time with a power meter, and write the difference.

Source: *Edge AI Engineering*, chapter “Setup”.
Published ranges with the active cooler.

	Short run	Long run
Duration	Some seconds	Minutes
State of the chip	Cold	At its final temperature
It answers	How fast is the board at its best?	How fast is the product all day?
Use it for	A comparison of two models	A product decision

- ▶ A run of MLPerf Inference has 600 seconds or more.
- ▶ Run until the temperature does not change. Report the latency of the first window and of the last window.
- ▶ State the cooling: the active cooler, a case, the temperature of the room. A board in a closed box is a different system.

Key point: A short benchmark shows the peak. The product gets the sustained value.

Source: The rules of MLPerf Inference (MLCommons).

Error	Result
The power in place of the energy	The slow board with low power looks better
The active power only	The battery estimate is too good: the rest state is missing
The chip value for a board	An error of a factor of 10 to 400 in the sleep
The peak current as the mean current	The battery estimate is too bad
No radio and no sensor in the budget	The largest consumers are missing
A power value before the warm-up	The power grows with the temperature
Energy with no accuracy and no window	You cannot compare the value with a second system

Key point: An energy value needs the same facts as a latency, and two more: the boundary and the instrument.

Part	You do	From the lecture
A	Write the protocol and your predictions	Parts 1 and 2
B	Four models on the kit: latency, flash, and arena	Part 2
C	The same models, MobileNetV2, and your detector on the Raspberry Pi, and a run of 3 minutes with the temperature	Parts 2 and 3
D	Energy for one inference, a battery estimate, and the report	Part 3

- ▶ The power of the kit: a USB power meter, or the data sheet values of this lecture.
- ▶ The power of the Raspberry Pi: the meter, or the estimate of the board.
- ▶ Each number of the report has its method. Each energy has its boundary.

Exercise: the battery life of a keyword detector

A keyword detector uses an ESP32-S3. For each window of audio it works for 50 ms: the features and the model. It has one window in each 250 ms.

Active power	217 mW
Power when the chip waits (idle)	157 mW
Power in the light sleep	0.79 mW
Battery	1000 mAh at 3.7 V

1. Calculate the duty cycle.
2. Calculate the mean power and the battery life if the chip waits between two windows.
3. Do the same for the light sleep between two windows.
4. The product must run for one month (720 hours). Which mean power is permitted? Which duty cycle gives it, with the light sleep?
5. Name two changes that can give this duty cycle, and one power consumer that is not in the table.

Answer: the battery life of a keyword detector

State between two windows	Mean power	Battery life
The chip waits	169.0 mW	21.9 hours
Light sleep	44.0 mW	84 hours: 3.5 days

1. $50/250 = 0.2$: a duty cycle of 20 percent.
2. $0.2 \times 217 + 0.8 \times 157 = 169.0$ mW. $3700/169.0 = 21.9$ hours.
3. $0.2 \times 217 + 0.8 \times 0.79 = 44.0$ mW. $3700/44.0 = 84$ hours.
4. $3700/720 = 5.14$ mW. $\alpha = (5.14 - 0.79)/(217 - 0.79) = 0.020$: a duty cycle of 2 percent, or 5 ms of work in each 250 ms.
5. Changes: a faster model (int8, a smaller network, better kernels); fewer windows; a small first stage that starts the model only when the sound is loud; an accelerator (0.03 mJ for one inference in Part 1). Not in the table: the microphone, the voltage regulator and the other parts of the board, and the radio.

- ▶ A benchmark looks at the system, at the model, and at the data. A standard suite such as MLPerf Tiny fixes the task, the dataset, and the run rules, so that two results compare.
- ▶ A fair measurement has a warm-up, many repetitions, and fixed inputs. Report the median and the 95th percentile of the latency, not only the mean.
- ▶ The time of the full pipeline is longer than the time of the model. A report says which of the two times it gives.
- ▶ The energy of one inference is the active power times the latency. The duty cycle gives the mean power, and the mean power gives the battery life. Record the temperature: a hot processor is slower.

Key point: A number with no method is not a result. For each number, give the device, the model file, the input, the warm-up, the number of repetitions, and the statistic.

Book chapters

- ▶ *Machine Learning Systems* (mlsysbook.ai), Volume I: chapter 12 “Benchmarking”.
- ▶ *Edge AI Engineering*: “Multi-Token Prediction on the Raspberry Pi 5”, section “How this was measured”.

Papers

- ▶ “MLPerf Tiny Benchmark”, Banbury et al., 2021.
- ▶ “MLPerf Inference Benchmark”, Reddi et al., 2020.
- ▶ “MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems from Microwatts to Megawatts for Sustainable AI”, Tschand et al., 2025.

Benchmark suites

- ▶ MLPerf Tiny: <https://mlcommons.org/benchmarks/inference-tiny/>
- ▶ MLPerf Inference Edge:
<https://mlcommons.org/benchmarks/inference-edge/>

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.